

评分：_____

南京信息工程大学

《面向对象程序设计实践》课程 综合实训报告

题 目 _____ 模拟教务系统 _____

学 院： _____ 计算机学院 _____

班 级： _____ 计科专业腾讯 1 班 _____

小组编号： _____ 第 2 组 _____

学 号： _____ 202383290121 _____

姓 名： _____ 梅应宝 _____

任课教师： _____ 耿兴华 _____

学 期： _____ 2025-2026 (1) _____

模拟教务系统

1 系统分析

1.1 应用系统简介

本课设面向高校教务业务场景，设计并实现一套“模拟教务系统”。系统围绕“课程—教师—学生—教学班—成绩”主业务链，支持学生选课、教师授课管理、教学班分配、成绩生成与查询等功能，旨在通过面向对象方法完成业务抽象、模块划分、类设计与数据库落地，并在可运行的软件原型中体现封装、继承、多态等核心思想。

系统在需求层面具有如下约束：

- (1) 业务边界：聚焦教务核心流程，不覆盖财务缴费、宿舍、图书等外围模块；
- (2) 数据一致性：选课记录、教学班分配与成绩结果必须在数据库中可追踪、可回溯；
- (3) 权限约束：学生、教师、教务/管理员角色的操作边界清晰；
- (4) 可扩展性：预留“学业监测/预警”扩展点，便于后期增量开发。

1.2 系统需求（功能）分析

结合模拟教务业务的典型流程，系统需求分为角色需求与公共需求两类：

1) 学生端需求

学生需要完成账号登录、课程信息浏览、按学期检索课程、提交选课/退课请求、查询个人课表、查询成绩与学分完成情况等。选课数据需落库，并与课程、教师、教学班等信息建立关联，保证后续分班与成绩生成的可操作性。

(2) 教师端需求

教师需要完成账号登录、查看本人可授课程/教学任务、选择授课课程并形成教学班、查看教学班学生名单、在教学结束后录入/导入成绩，并支持按教学班导出成绩清单。教师端的数据输出应能与学生端成绩查询闭环联动。

(3) 教务/管理员端需求

教务端需要维护基础数据（学生、教师、课程、专业等）、配置开课计划、组织选课、执行教学班分配（例如依据容量上限或规则分班），并在学期末汇总成绩。为提升系统完整性，可提供基本统计功能（课程选课人数、课程通过率等）以及数据导入导出接口。

(4) 公共需求

系统应提供统一的身份认证与权限控制、关键操作日志（选课、分班、成绩生成/修改）、异常处理与输入校验，并保证数据库操作的原子性（必要时采用事务），以降低数据不一致风险。

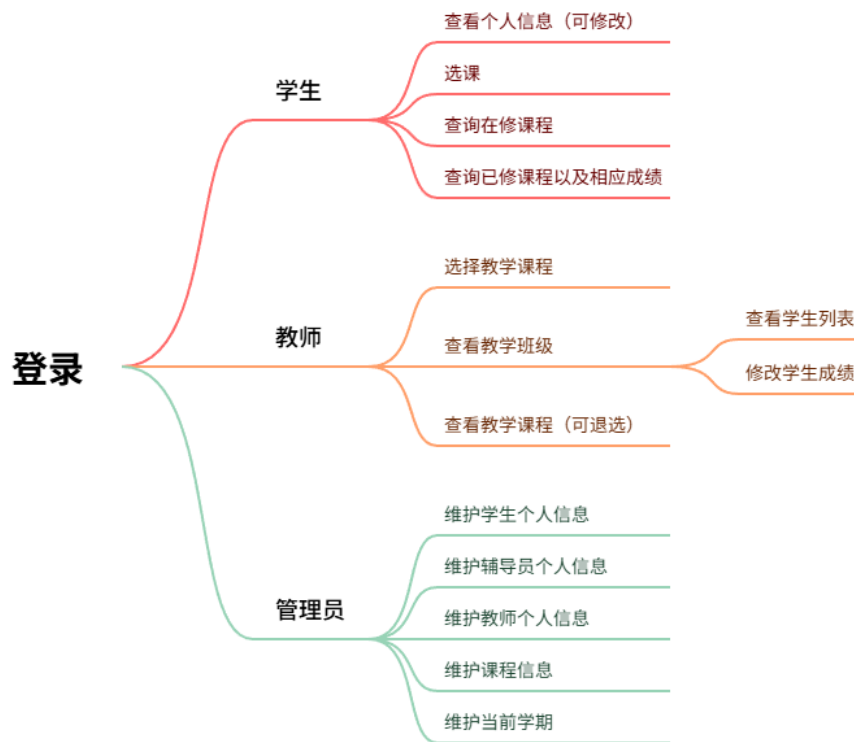
2 系统设计

2.1 总体设计

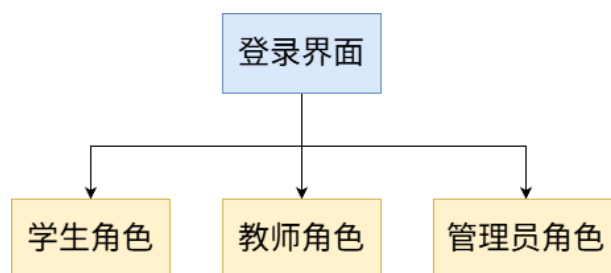
系统采用“分层 + 模块化”的总体设计思路,将复杂业务按职责拆解为若干相对独立的功能单元,并通过统一的数据模型与清晰的接口实现模块协作与解耦。整体架构划分为四层:

1. **表现层 (UI/交互)**: 负责界面展示、输入校验与交互反馈;
2. **业务层 (Service/规则)**: 负责业务流程编排、规则校验与权限控制;
3. **数据访问层 (DAO/Repository)**: 封装数据库读写与查询逻辑;
4. **数据层 (MySQL 数据库)**: 存储基础数据与业务数据,支撑持久化与一致性。

功能模块划分 (与界面入口及交互操作一一对应):

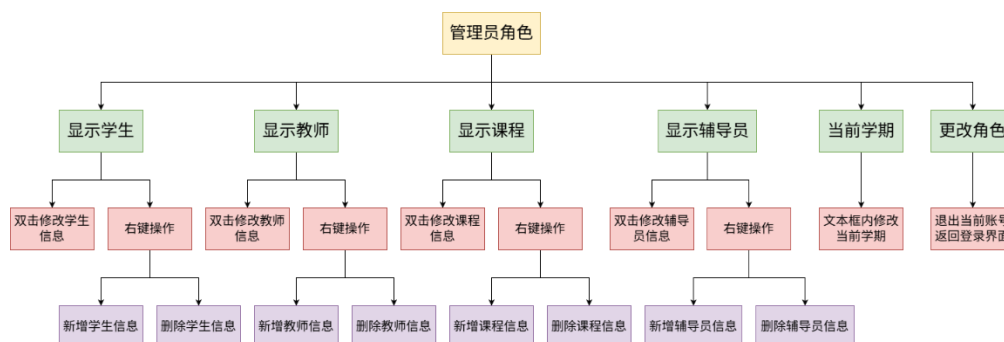


1) **用户与权限模块**: 提供登录入口与身份选择 (学生/教师/管理员); 完成角色识别、权限校验与会话管理; 支持切换用户、退出返回登录; 管理员端提供角色调整/权限变更入口, 确保不同角色仅能访问授权功能。

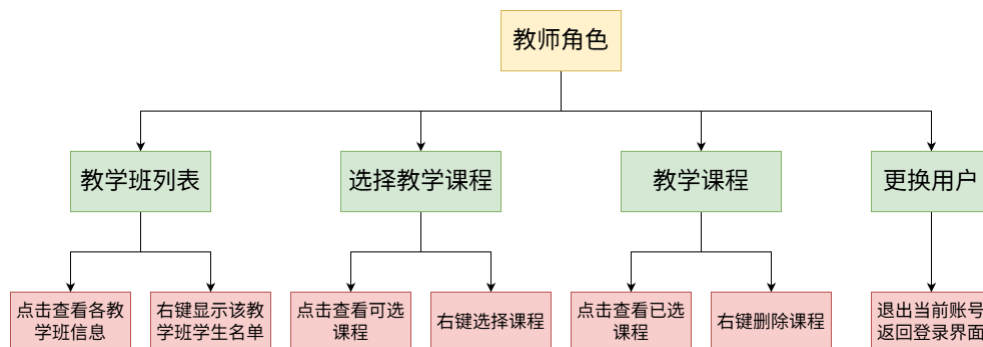


2) 基础信息查询展示模块: 对学生、教师、课程、辅导员等基础信息提供统一的列表展示与明细查看能力，对应界面“显示学生/显示教师/显示课程/显示辅导员”等功能入口，便于各角色快速获取全局数据。

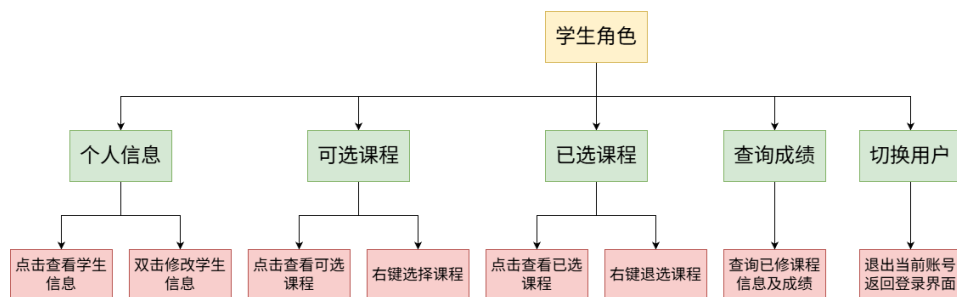
3) 基础数据维护模块 (管理员): 面向管理员提供学生/教师/课程/辅导员数据的新增、删除、修改；支持双击编辑字段与右键菜单快捷增删改；数据变更实时写入数据库并同步刷新界面，保障维护效率与数据一致性。



4) 教学班与授课管理模块 (教师): 支持教师选择授课课程、生成与维护教学班；提供教学班列表查看、按班级查看学生名单等操作；支持已选授课课程的查询与删除，为教务分班、名单维护等流程提供支撑。



5) 选课管理模块 (学生): 支持课程检索与可选课程浏览；提供选课/退课（右键操作）；在业务层完成选课冲突校验与课程容量校验；选课结果持久化到数据库，并与“可选课程/已选课程”视图联动更新。



6) 成绩管理模块: 学生端提供成绩查询（已修课程与成绩展示）；教师/教务端支持成绩录入与批量导入；提供成绩统计分析与导出接口，便于后续扩展报表与数据分析功能。

2.2 详细设计

2.2.1 数据结构设计

本系统采用 MySQL 关系型数据库对教务业务数据进行统一管理。围绕高校教务管理的核心流程，数据库设计覆盖学生、教师、课程、教学班、选课与成绩等关键实体，并通过主外键约束与触发器机制保证数据一致性。

在选课与成绩管理方面，系统设计了 `courseSelection` 表，用于记录学生在不同学期、不同修读状态下的课程选择情况。通过引入 `FirstRepair` 字段，实现了对首次修读与重修课程的区分，使系统能够真实反映学生的学习过程。

在教学组织方面，系统采用 `teacher_course` 与 `teach_class` 两级结构对教师授课行为进行建模。其中，`teacher_course` 表用于描述教师在某一学期承担某门课程的教学任务；`teach_class` 表则作为具体教学班实例，记录课程安排与教学信息。为减少业务层复杂度并避免数据不一致，系统通过数据库触发器在 `teacher_course` 插入时自动生成对应教学班，从而实现教学班的自动化管理。

1) student（学生基础信息表）

主键：**student_uid**（**CHAR(12)**），学号唯一标识。

student_uid	学号 / 唯一 ID（主键，NOT NULL）
name	姓名（VARCHAR(128)）
major	专业（VARCHAR(128)）
telephone	手机号（VARCHAR(32)）
wechat_id	微信号（VARCHAR(64)）
email	邮箱（VARCHAR(255)）

2) teacher（教师基础信息表）

主键：**teacher_uid**（**CHAR(10)**），教师工号唯一标识。

teacher_uid	教师工号 / 唯一 ID（主键，NOT NULL）
-------------	---------------------------

name	姓名 (VARCHAR(128))
telephone	手机号 (VARCHAR(32))
wechat_id	微信号 (VARCHAR(64))
email	邮箱 (VARCHAR(255))

3) counselor (辅导员基础信息表)

主键: **counselor_uid (CHAR(10))**, 辅导员工号唯一标识。

counselor_uid	辅导员工号 / 唯一 ID (主键, NOT NULL)
name	姓名 (VARCHAR(128))
telephone	手机号 (VARCHAR(32))
wechat_id	微信号 (VARCHAR(64))
email	邮箱 (VARCHAR(255))

4) course (课程基础信息表)

主键: **course_uid (CHAR(10))**, 课程编号唯一标识。

course_uid	课程编号 / 唯一 ID (主键, NOT NULL)
course_name	课程名称 (VARCHAR(128))
credits	学分 (INT)
category	课程类别 (VARCHAR(64))

5) user_account (统一登录账号表)

主键: **username (VARCHAR(64))**, 系统登录用户名。

username	登录用户名 (主键, NOT NULL)
password	登录密码 (CHAR(32), MD5 加密存储)
role	用户角色 (VARCHAR(32))
created_at	账号创建时间 (DATETIME, 默认当前时间)

6) teacher_course (教师授课任务表)

主键: **(teacher_uid, course_uid, semester)**, 联合主键。

teacher_uid	教师工号 (外键, 关联 teacher 表)
course_uid	课程编号 (外键, 关联 course 表)
semester	学期 (VARCHAR(16))

7) teach_class (教学班表)

主键: **class_id (CHAR(10))**, 教学班唯一标识。

class_id	教学班编号（主键，NOT NULL）
course_uid	课程编号（外键，关联 course 表）
teacher_uid	教师工号（外键，关联 teacher 表）
semester	学期（VARCHAR(16)）
schedule	上课时间（VARCHAR(64)）
location	上课地点（VARCHAR(64)）

8) courseSelection（选课与成绩过程表）

主键：（student_uid, course_uid, FirstRepair），联合主键。

student_uid	学号（外键，关联 student 表）
course_uid	课程编号（外键，关联 course 表）
selection_date	选课时间（DATETIME）
grade	成绩（DECIMAL(5,2)，可为空）
FirstRepair	首次/重修标识（CHAR(1)，0 为首次，1 为重修）

9) 约束与触发器汇总

一致性约束：全库采用外键将学生/教师/课程等主数据与授课任务、教学班、选课过程强关联，满足“选课—分班—成绩”全链路可追踪目标。

自动化策略：teacher_course 插入后由触发器自动创建 teach_class，避免业务层重复写入与不一致风险。

删除联动策略：对 student/teacher/counselor 的删除使用 BEFORE DELETE 触发器清理从表数据，避免外键冲突并保证账号同步清理。

2.2.2 主要类设计

本系统在实现过程中采用面向对象方法对教务业务进行建模，根据系统功能与代码结构，将主要类划分为**界面控制类（UI/Controller）**、**业务与数据访问类（DAO/Helper）**、**全局配置与工具类**三类。各类之间职责边界清晰，通过接口调用完成协作，从而提高系统的可维护性与扩展性。

（1）界面控制类（UI/Controller 类）

界面控制类主要负责用户交互、界面展示以及对用户操作的响应，其内部通过调用数据访问类完成数据库读写操作，而不直接处理底层数据库细节。

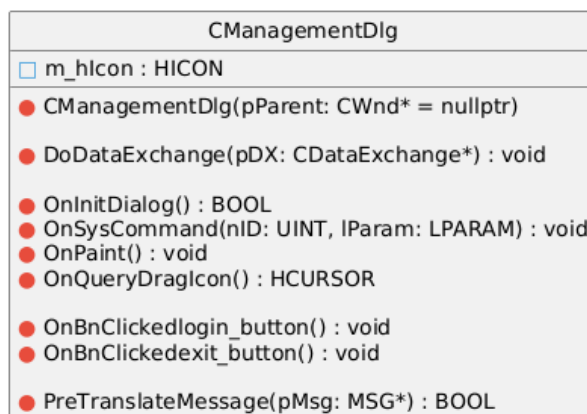
1) CManagementDlg（系统登录与角色分流类）

该类作为系统入口界面，负责完成用户身份认证与角色识别。登录过程中，系统根据用户输入的用户名与密码，查询用户账户表以获取对应角色信息，并根据角色类型打开相应功能界面。该类实现了系统的统一登录控制，是各功能模块的调度中心。

该类主要职责包括：

- 1.接收用户登录信息并完成合法性校验；
- 2.查询用户账户表以确认用户角色；
- 3.根据角色类型分流至学生端、教师端、辅导员端或管理员端界面；
- 4.提供基础的键盘事件处理（如回车触发登录）。

其功能实现与数据库中 `user_account` 表保持一致，通过角色字段区分不同用户权限层级，从而实现系统的权限控制。



2) StudentDlg（学生端功能类）

该类用于实现学生用户的主要业务功能，包括课程浏览、选课与成绩查询。学生在该界面中可查看系统中已开放课程，并根据自身需求进行选课或退课操作。选课结果将被写入数据库选课记录表，供后续教学班分配与成绩管理使用。

该类主要职责包括：

- 1.查询并展示课程信息；
- 2.提交选课请求并写入选课记录；
- 3.提交退选请求并删除相应选课记录；
- 3.查询并展示学生个人已获得成绩信息；
- 4.更新个人信息；
- 5.通过界面事件触发数据库操作。

该类与数据库中的 `courseSelection` 表直接对应，选课操作通过插入选课记录实现，成绩查询则通过读取选课记录中的成绩字段完成。

C StudentDlg
m_list : CListCtrl m_dbHelper : CDatabaseHelper m_currentTable : CString editItem : CEdit hitRow : int hitCol : int m_allowUnselectInCourse : bool
StudentDlg(CWnd* pParent = nullptr) ~StudentDlg() DoDataExchange(CDataExchange* pDX) OnClickedPersonShow() OnInitDialog() : BOOL OnDbcList(NMHDR* pNMHDR, LRESULT* pResult) PreTranslateMessage(MSG* pMsg) : BOOL OnEditKeyDown(UINT nChar, UINT nRepCnt, UINT nFlags) OnEditKillFocus() OnBnClickedChooseClass() OnBnClickedcourses() OnListRClick(NMHDR* pNMHDR, LRESULT* pResult) InsertCourseSelectionAt(CListCtrl& listCtrl, CDatabaseHelper& dbHelper, int nItem, const CString& studentUidParam, const CString& selectionDateParam) : bool DeleteCourseSelectionAt(CListCtrl& listCtrl, CDatabaseHelper& dbHelper, int nItem, const CString& studentUidParam, const CString& selectionDateParam) : bool OnBnClickedStuChange() OnBnClickedSearchGrade()

3) teacherdlg（教师端功能类）

该类用于实现教师端的教学管理功能，主要包括授课课程选择、教学班查看以及学生名单管理等。教师在选择授课课程后，系统会自动生成对应教学班，教师可在界面中查看本人所负责的教学班信息。

类主要职责包括：

- 1.展示教师可授课程列表；
- 2.提交教师授课选择；
- 3.提交取消授课课程；
- 4.查询并展示对应教学班信息；
- 5.查看教学班内学生名单；
- 6.登记教学班内学生成绩。

教师选择授课课程后，系统在数据库中插入授课记录，并通过数据库触发器机制自动生成教学班，从而实现教学组织过程的自动化。该类的业务实现与 teacher_course 表、courseSelection 表及 teach_class 表紧密关联。

C teacherdlg
m_dbHelper : CDatabaseHelper m_list : CListCtrl m_teacherUid : CString m_currentTable : CString
teacherdlg(CWnd* pParent = nullptr) ~teacherdlg() SetTeacherUid(const CString& uid) GetTeacherUid() const : CString GetCurrentTable() const : CString DoDataExchange(CDataExchange* pDX) OnBnClickedshowclass() OnLvniItemchangedworkpanel(NMHDR* pNMHDR, LRESULT* pResult) OnListRClick(NMHDR* pNMHDR, LRESULT* pResult) OnShowClassStudents() OnBnClickedChoosecourse() OnBnClickedclasses() OnBnClickedchangeuser()

4) class_students (教学班学生列表 / 成绩与选课记录编辑类)

该类用于在教师端以“教学班/课程”为入口，展示该课程在指定学期内的选课学生记录，并提供面向表格数据的可视化编辑能力。。

该类主要职责包括：

1.在教师端需要查看教学班学生名单时，提供统一的“学生选课记录列表”展示界面，通过 `course_uid` 过滤选课记录，实现按课程维度的学生汇总展示；

2.支持按学期过滤数据：优先使用外部传入的 `semester`（通常来自教学班记录），若未传入则回退使用系统全局当前学期；并将“YYYY-1 / YYYY-2”学期格式映射为时间区间，对 `selection_date` 进行范围筛选，以保证展示集合与教学班学期一致；

3.通过 `CListCtrl` 的双击事件实现单元格编辑：用户双击某一行/列后在对应位置弹出编辑框，支持直接修改列表中的字段值，提高教师端维护效率；

4.在编辑提交时进行“唯一定位记录”的安全更新：通过读取列表表头得到字段名，并使用复合主键字段（`student_uid + course_uid + FirstRepair`）构造 `WHERE` 条件，确保仅更新唯一一条选课记录，避免误更新整表数据；

5.对输入值进行基础防护处理：例如对单引号进行转义，降低 `SQL` 拼接导致的语句错误风险；同时在缺失关键主键列或主键值不完整时拒绝修改，保证更新操作的可控性与数据一致性。

通过引入该类，系统将“教学班学生数据展示”与“选课记录可视化维护”集中在一个独立界面中实现，使 `teacherdlg` 等业务界面类更专注于流程控制（如选择授课课程、进入教学班视图），而将表格展示、双击编辑、唯一键定位更新等通用交互逻辑封装在专用类中。该设计在工程上提升了教师端的数据可操作性，并且通过复合主键约束与学期口径过滤机制，保证了对 `courseSelection` 数据读写的一致性与可追溯性。

class_students				
m_pDb : CDatabaseHelper* = nullptr m_courseUid : CString m_semester : CString m_studentsList : CListCtrl editItem : CEdit hitRow : int = -1 hitCol : int = -1 <tr><td> class_students(pParent: CWnd* = nullptr) ~class_students() <tr><td> SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr></td></tr></td></tr>	class_students(pParent: CWnd* = nullptr) ~class_students() <tr><td> SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr></td></tr>	SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr>	DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr>	OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void
class_students(pParent: CWnd* = nullptr) ~class_students() <tr><td> SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr></td></tr>	SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr>	DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr>	OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void	
SetDatabaseHelper(pDb: CDatabaseHelper*) : void SetCourseUid(uid: CString&) : void SetSemester(sem: CString&) : void <tr><td> DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr></td></tr>	DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr>	OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void		
DoDataExchange(pDX: CDataExchange*) : void OnInitDialog() : BOOL PreTranslateMessage(pMsg: MSG*) : BOOL <tr><td> OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void </td></tr>	OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void			
OnLvnItemchangedlist(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void				

5) manager (管理员端功能类)

管理员端用于维护系统基础数据，是系统数据管理的核心界面。管理员可对学生、教师、课程及辅导员等基础信息进行增删改查操作，并可配置系统当前学期参数。

该类主要职责包括：

- 1.管理学生、教师、课程、辅导员等基础信息；
- 2.执行数据的新增、修改与删除操作；
- 3.配置系统学期信息，为选课与授课提供时间基准。

该类对应数据库中的多张基础信息表，并通过数据库触发器机制保证在删除操作时相关数据的完整性与一致性。

manager
m_list : CListCtrl m_dbHelper : CDatabaseHelper m_currentTable : CString = "无" editItem : CEdit m_semesterEdit : CEdit hitRow : int hitCol : int
manager(pParent: CWnd* = nullptr) ~manager() DoDataExchange(pDX: CDataExchange*) : void OnBnClickedstudent_show() : void OnBnClickedteacher_show() : void OnBnClickedcourse_show() : void OnBnClickedcounselor_show() : void OnListRClick(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnMenuAdd() : void OnMenuDelete() : void OnDbClickList(pNMHDR: NMHDR*, pResult: LRESULT*) : void OnEditKillFocus() : void OnEditKeyDown(nChar: UINT, nRepCnt: UINT, nFlags: UINT) : void OnEnChangeSemesterEdit() : void OnBnClickedchangeid() : void PreTranslateMessage(pMsg: MSG*) : BOOL OnInitDialog() : BOOL SaveSemesterEdit() : void

(2) 数据访问与辅助类 (DAO / Helper 类)

1) CDatabaseHelper (数据库访问辅助类)

该类是系统中用于封装数据库操作的核心类，负责统一管理数据库连接、SQL 语句执行以及结果集处理。系统中各界面控制类均通过该类完成数据库访问，从而避免在业务代码中直接操作数据库接口。

该类主要职责包括：

- 1.建立与维护数据库连接；
- 2.执行查询与更新类 SQL 语句；
- 3.封装通用的数据插入、修改与删除操作；
- 4.将查询结果转换为界面可展示的数据结构。

通过引入该类，系统实现了界面逻辑与数据访问逻辑的分离，提高了代码复用性与系统整体可维护性。

CDatabaseHelper	
m_pConnection : MYSQL* m_bConnected : BOOL	
CDatabaseHelper() ~CDatabaseHelper()	
Connect(strHost: CString& = "localhost", strUser: CString& = "root", strPassword: CString& = mysql_password, strDatabase: CString& = "Schooldb", nPort: int = 3306) : BOOL Disconnect() : void IsConnected() const : BOOL	
DisplayTableData(listCtrl: CListCtrl&, strTableName: CString&, strColumns: CString& = "*", strWhere: CString& = "") : BOOL	
InsertRecord(strTableName: CString&, arrFields: CStringArray&, arrValues: CStringArray&) : BOOL UpdateRecord(strTableName: CString&, strWhere: CString&, arrFields: CStringArray&, arrValues: CStringArray&) : BOOL DeleteRecord(strTableName: CString&, strWhere: CString&) : BOOL	
Utf8ToCString(utf8Str: char*) : CString CStringToUtf8(str: CString&) : CStringA	
ExecuteQuery(strSQL: CString&) : BOOL EscapeString(strValue: CString&) : CString	

2) InputDlg 类（通用输入对话框类）

该类用于封装系统中的通用输入行为，主要解决系统中多个界面在数据维护、信息录入过程中频繁需要弹出输入对话框并获取用户输入的问题。

该类主要职责包括：

- 1.在系统需要用户输入数据时，提供统一的输入对话框界面，通过弹出方式获取用户输入内容，避免在不同业务界面中重复设计输入控件；
- 2.利用 MFC 的 DDX 数据交换机制，将输入控件内容与成员变量进行绑定，实现界面数据与程序数据之间的自动同步；
- 3.通过可配置的提示信息机制（Hint），在对话框中动态显示当前输入字段的说明，提高用户输入的准确性与友好性；
- 4.在用户确认输入时，统一获取并返回输入结果，由调用界面类进一步触发相应的业务逻辑或数据库更新操作；
- 5.配合界面控制类实现数据维护与编辑操作，使业务界面类专注于流程控制而非输入细节处理。

通过引入 InputDlg 类，系统将“用户输入交互”从 StudentDlg、teacherdlg、manager 等具体业务界面类中抽离出来，实现了输入逻辑的集中管理。该设计有效降低了界面类之间的耦合度，使各业务界面能够更加专注于自身功能实现，体现了面向对象设计中封装性与单一职责原则的思想。

InputDialog	
☐	m_strInput : CString
☐	m_strHint : CString
☒	InputDialog(pParent: CWnd* = nullptr)
☒	~InputDialog()
☒	DoDataExchange(pDX: CDataExchange*) : void
☒	OnBnClickedOk() : void
☒	SetHint(hint: CString&) : void
☒	OnInitDialog() : BOOL

(3) 全局配置与工具类

① Global（全局配置类）

该类用于存储系统运行过程中需要共享的全局变量，如当前学期信息、数据库连接参数等。通过集中管理全局配置，避免了参数在多处重复定义的问题。

② tool（工具类）

该类提供若干通用工具函数，用于界面控件操作、字符串处理等辅助功能，减少了界面控制类中的冗余代码。

(4) 类设计与数据库结构的对应关系说明

系统主要类与数据库表之间保持一一对应或一对多的关系：

- 1.登录与权限控制类对应用户账户表；
- 2.学生与教师功能类对应选课表、授课表及教学班表；
- 3.管理员功能类对应基础信息表；
- 4.数据访问辅助类统一负责所有数据库表的操作。

通过上述设计，系统在类结构层面与数据库结构层面形成了清晰、稳定的映射关系，为系统的功能实现与后期扩展奠定了良好基础。

3 系统实现

3.1 系统开发环境

本系统在 Windows 平台下完成开发与调试，采用 C++ 语言结合图形化界面框架实现教务管理功能，并使用 MySQL 关系型数据库作为后端数据存储。系统整体开发环境配置如下。

在操作系统方面，系统运行于 Windows 桌面环境下，依托 **Visual Studio** 提供的集成开发环境完成程序的编译、调试与运行。开发过程中使用 Visual Studio 的 **MFC (Microsoft Foundation Classes)** 框架构建图形用户界面，通过对话框形式实现登录界面及学生端、教师端、管理员端等功能界面，从而提升系统的交互性与可操作性。

在程序设计语言方面，系统核心逻辑采用 C++ 语言实现。通过面向对象方法对教务业务进行建模，将系统划分为界面控制类、数据访问类以及辅助工具类等模块，各模块之间职责清晰，便于维护与扩展。程序使用 Visual Studio 工程文件（.sln、.vcxproj）进行组织，支持多源文件协同编译。

在数据库方面，系统采用 MySQL 作为后端数据库管理系统，用于存储学生、教师、课程、选课、教学班及用户账户等核心数据。数据库字符集统一设置为 utf8mb4，以保证对中文信息的良好支持。系统通过 MySQL 提供的 C API 与数据库进行通信，实现数据的增删改查操作。数据库结构及示例数据通过统一的初始化脚本完成创建与导入，便于系统部署与测试。

在数据管理与调试方面，开发过程中可借助 MySQL 图形化管理工具对数据库表结构、数据内容及执行结果进行检查与验证，以辅助程序功能调试和测试验证。数据库脚本中预置了多角色用户账号及示例选课、授课与成绩数据，为系统功能测试提供了完整的数据基础。

在版本管理方面，系统源代码通过 Git 进行集中管理，便于团队成员协同开发与版本迭代。通过版本控制工具，能够有效追踪代码修改历史，降低多人协作过程中产生的冲突风险。

Github 仓库：[MEIYBAO/OOD-Project: 面向对象程序设计课程设计](https://github.com/MEIYBAO/OOD-Project)。

3.2 程序编码

阐述系统的开发过程，给出核心的代码。（宋体，5 号，段落首行缩进 2 字符）

系统编码遵循“先抽象后实现”的思路：先根据需求确定实体与用例，再完成数据库表结构设计，随后实现 DAO 层与 Service 层，最后对接界面交互并贯通测试。

核心代码建议在报告中呈现 2—4 段“最能体现课程价值”的片段（无需铺满代码）：

- （1）数据库连接与统一执行封装（体现封装与异常处理）；
- （2）选课接口的业务校验（体现职责分离：校验在 Service，CRUD 在 DAO）；
- （3）分班策略函数（体现策略/规则）；
- （4）成绩录入与查询的关键 SQL 或接口（体现数据一致性）。

4 系统测试

以系统的某一项（或几项）功能为例，介绍系统的操作流程和相应结果。（宋体，5 号，段落首行缩进 2 字符，附图居中排版）

以“学生选课—教务分班—教师录入成绩—学生查成绩”作为联通测试用例。首先，使用初始化脚本导入基础数据（学生、教师、课程等），随后学生端登录并选择某学期课程，系统提示选课成功并在数据库中生成选课记录；接着教务端执行分班操作，系统按容量规则生成教学班并将选课学生写入班级成员关系；最后教师端进入教学班并录入成绩，学生端重新登录后可在成绩查询页面查看对应课程的成绩结果，与数据库中的 Score 记录一致。

5 总结与体会

系统开发过程中遇到的问题及解决的方法，应用系统的优点和不足，以及实训的收获、体会等。（宋体，5号，段落首行缩进2字符）

本项目将教务核心业务抽象为若干对象与服务，通过实体类刻画业务数据，通过 Service 层组织业务规则，通过 DAO 层隔离数据库访问，形成了结构清晰、可扩展的系统原型。开发过程中主要挑战集中在三方面：其一是业务数据的一致性维护（选课、分班、成绩三者关联），通过规范主外键设计与必要的事务控制降低了不一致风险；其二是角色权限边界的梳理，通过统一认证与权限校验避免了越权操作；其三是系统可扩展性，在总体设计阶段预留了“学业监测”模块接口，为后续迭代提供了结构基础。

不足之处在于：当前系统仍是模拟场景，复杂规则（如先修课、时间冲突的精细化、并发选课一致性、成绩导入模板校验）有待完善；界面交互与异常提示也可进一步优化。通过本次实训，我对面向对象方法在真实业务建模、分层设计、类职责划分与数据库协同方面有了更系统的理解，并掌握了从需求分析到实现与测试的完整工程流程。

参考文献： 3 篇左右，按要求书写

[1] 仿宋，5号

[2]

[3]

小组编号	*班第*组	组长	202383290121 梅应宝
学号	姓名	具体任务（分工）	
202383290121	梅应宝	系统框架搭建，数据库搭建，登录窗口实现，管理员窗口实现	
202383290382	董建标	系统框架搭建，教师窗口实现	
202383290102	丁诚诚	系统框架搭建，学生窗口实现	